This the complete conversation with Copilot; we also asked Gemini in some little details since the Copilot code written here is not 100% correct. Examples include that time, choice are reserved words cannot be used for type names; queries must be defined before process (process is kind of like the main of a program)

Conversation

You picked a perfect protocol to formalize—there's a lot here that *can* be pushed into tools, and a hard core (court reasoning) that will stay partly "on paper". Let's structure it so you can actually start coding models, not just talk about them.

"In this paper, we propose an in-booth e-voting solution that allows voters to challenge their individual votes in court and allows judges to verify by themselves without calling cryptographic experts. The proposed system uses 2 devices; the first device authenticates the voter and prints a receipt with a unique cryptographic ID code (HBID) for each voter. The second is a voting device that permits successive multiple voting and prints a receipt for each voting attempt; the device then prints a sealing receipt for poll workers with enough information to identify the voter's final receipt."

---

1. Decide what to verify, formally

Automated tools will handle some properties very well, others only in an abstracted way.

Good candidates for automated verification

- **No double sealing receipt per HBID**
- **No double voting counted** (HBID uniqueness, final-vote flag)
- **Recorded-as-cast / counted-as-recorded** (link between device2 final vote and PBB entry)
- **Resistance to malicious device2** (Benaloh-style challenge correctness)
- **Non-linkability of HBID to ID** (modeled as hash with randomness)

Harder, but still partially formalizable

- **Coercion-resistance** (esp. with multiple receipts and old receipts)
- **Court verifiability / dispute resolution** (Table 2 scenarios, judge reasoning)

These "court" aspects can be reduced to *protocol-level* properties like: "if EA presents a sealing receipt consistent with PBB, then there exists a unique underlying cast event", which tools can check as correspondence/injectivity properties.

---

2. Choose a tool split: symbolic vs. state-level

I'd strongly suggest a **two-track** approach:

- **Symbolic crypto & security properties:**
  ProVerif or **Tamarin** for:
  - HBID as randomized hash
  - Receipts as messages
  - Benaloh challenge behavior
  - Privacy and verifiability queries (in the style of Cheval–Cortier–Debant's "Election Verifiability with ProVerif"). Inria
- **State-machine & consistency properties:**
  TLA+ / PlusCal (or Ivy) for:
  - Device1/Device2 state, memory cards, counters
  - Checkpoint summary reports and PBB updates
  - Invariants: "each HBID has at most one sealing receipt", "PBB equals aggregation of final votes", "summary subtotals match PBB per booth/interval".

This mirrors how recent work separates *cryptographic* verifiability from *system* correctness. Inria

---

3. Abstract protocol model (for both lines)

At a high level, you can model:

- **Roles**
  - **Voter**: interacts with device1, then device2, may repeat votes, chooses final.
  - **Device1**: authenticates, generates HBID, stores `(ID, HBID, bio, time, booth)` and prints receipt1 + passing ticket.
  - **Device2**: accepts HBID once, allows multiple votes, prints receipt2 for each, and one sealing receipt3 for poll workers.
  - **EA / Voting server**: aggregates memory cards, checks consistency, publishes PBB.
  - **PBB**: public append-only board of `(booth, time, choice, no_of_votes)`.
  - **Adversary**: Dolev–Yao on public channels, may corrupt device2, poll workers, etc.
- **Key abstractions**
  - `HBID = H(ID, r)` with `r` fresh; treat `H` as a cryptographic hash with no inverse equations.
  - Receipts as tuples:
    - `R1 = (ID, HBID, time1, booth)`
    - `R2 = (HBID, f(HBID, choice), count, time2, booth)`
    - `R3 = (HBID, f(HBID, choice_final), final_count, time3, booth)` (poll worker sealing receipt).
  - PBB entries: `(booth, time3, choice_final, no_mult_votes)`.

4. ProVerif line: how to encode and what to ask

4.1. Basic skeleton

You can start with a ProVerif-style process model (pseudo-code):

```
(* Types and functions *)
type id.
type hbid.
type choice.
type time.
type booth.
type receipt1.
type receipt2.
type receipt3.

fun H(id, nonce): hbid.
fun f(hbid, choice): bitstring.

(* Events *)
event cast_final(hbid, choice, time, booth).
event stored_PBB(booth, time, choice, count).
event sealing_receipt(hbid, choice, time, booth).

(* Voter process (simplified) *)
process
  new ID: id;
  new r: bitstring;
  let HB = H(ID, r) in
  (* device1 interaction abstracted *)
  event receipt1_generated(ID, HB);
  (* device2: multiple votes, we keep only final *)
  new t_final: time;
  new c_final: choice;
  event cast_final(HB, c_final, t_final, someBooth);
  event sealing_receipt(HB, c_final, t_final, someBooth);
  (* EA later produces stored_PBB *)
  0
```

Then add an EA process that, upon `sealing_receipt`, eventually emits `stored_PBB(booth, time, choice, count)`.

4.2. Core queries

- **No double sealing receipt per HBID**

```
query inj-event(sealing_receipt(h, c, t, b)) ==> inj-event(cast_final(h, c, t, b)).
```

- **Recorded-as-cast / counted-as-recorded**

```
query event(cast_final(h, c, t, b)) ==> event(stored_PBB(b, t, c, _)).
```

- **Privacy of choice vs. adversary**

Model two elections where a given voter swaps `choice0` and `choice1`, and ask observational equivalence (or use the Cheval–Cortier–Debant characterization of E2E-verifiability to reduce to simpler properties). Inria

- **Benaloh challenge correctness**

Model a branch where the voter "challenges" a trial vote: the device must reveal randomness that allows recomputation of the ciphertext/receipt; ask a correspondence between "challenge accepted" and "ciphertext consistent with choice".

4.3. Court-verifiability as correspondence

You can encode a simplified "judge" view:

- If a judge sees:
  - voter receipt `R2_final` (with `HBID`, `f(HBID, choice)`, `time3`, `booth`),
  - EA sealing receipt `R3` for same `HBID`,
  - and a PBB entry `(booth, time3, choice, count)`,
- then there exists a unique `cast_final(HBID, choice, time3, booth)` event.

This becomes an injective correspondence query tying together the three evidence sources.

---

5. TLA+ line: system invariants and dispute scenarios

Here you ignore crypto and treat HBIDs as unique opaque IDs.

5.1. State variables

- `Device1Log ⊆ [ID, HBID, Time1, Booth]`
- `Device2Log ⊆ [HBID, Choice, Count, Time2, Booth, Final?]`
- `SealingBox ⊆ [HBID, Choice, FinalCount, Time3, Booth]`
- `PBB ⊆ [Booth, Time3, Choice, NoMultVotes]`
- `SummaryReports ⊆ [Booth, Interval, NoVoters, NoVotes, MerkleRootHBIDs, MerkleRootVotes]`

5.2. Key invariants

- **HBID uniqueness:**
  For all distinct entries in `Device1Log`, `HBID` differs.
- **Single sealing receipt per HBID:**
  For each `HBID`, at most one element in `SealingBox` with that `HBID`.

- **Final vote consistency:**
  For each `HBID`, the unique `Final? = TRUE` entry in `Device2Log` matches the sealing receipt and the PBB entry.
- **Summary consistency:**
  For each booth/interval, the counts in `SummaryReports` equal the aggregation of `Device1Log`/`Device2Log` restricted to that interval.

5.3. Dispute resolution scenarios

You can encode the Table 2 cases as temporal properties:

"If EA brings a sealing receipt and a matching record, and the voter claims 'I didn't vote', then either the biometric/time evidence contradicts the voter's claim, or the system is in an error state."

In TLA+, that becomes: under assumptions about honest devices, there is no reachable state where EA presents consistent evidence while the voter never executed a `cast_final` action.

---

6. Concrete next steps

If you'd like, we can:

1. Pick **one property** (e.g., "no double sealing receipt per HBID + recorded-as-cast") and I'll help you write a *full* ProVerif model skeleton around it.
2. In parallel, sketch a **minimal TLA+ spec** with Device1/Device2/EA and one invariant, so you have a runnable model checker setup.

Which property do you want to formalize first—privacy, recorded-as-cast, or the court-verifiability correspondence?

Great—ProVerif is a very natural fit for the "court verifiability" core.

"Our proposed design aims to achieve what we call complete and sound court verifiability; defined as: the court ability to decide the integrity of election, without expert knowledge, by using receipts with tracking codes and public bulletin boards."

Below is a **minimal but coherent ProVerif skeleton** focused only on court verifiability. We can refine it once you're happy with the structure.

---

1. High-level modeling choices

We abstract:

- HBID as a randomized hash of the voter's identity.
- Receipts as events plus messages; we don't model printing, just the fact they exist.
- EA and devices as honest for now (we'll add adversarial behavior later).
- Court verifiability as a **correspondence** between:
  - A "judge accepts evidence" event, and
  - A unique underlying `cast_final` event.

We ignore coercion and Benaloh randomness here; only the final vote and receipts matter.

---

2. Core types, functions, and events

```
(* Types *)
type id.
type hbid.
type choice.
type time.
type booth.
type fval.      (* value of f(HBID, choice) printed on receipts *)

(* Cryptographic primitives *)
fun H(id, bitstring): hbid.      (* randomized HBID generator *)
fun F(hbid, choice): fval.       (* simple function printed on receipts *)

(* Events: ground truth *)
event cast_final(hbid, choice, time, booth).
event sealing_receipt(hbid, choice, time, booth).
event pbb_entry(booth, time, choice, bitstring).  (* count abstracted *)

(* Events: what the court sees *)
event voter_receipt(hbid, fval, time, booth).
event ea_receipt(hbid, choice, time, booth).
event judge_accepts(hbid, choice, time, booth).
```

---

3. Honest voter, devices, and EA (simplified)

```
(* One honest voter, one booth, one final vote *)
process
  new ID : id;
  new r  : bitstring;
  let HB = H(ID, r) in
  new T  : time;
  new B  : booth;
  new C  : choice;

  (* Device2 final vote *)
  event cast_final(HB, C, T, B);

  (* Voter's final receipt: HBID, F(HB,C), time, booth *)
  let FV = F(HB, C) in
  event voter_receipt(HB, FV, T, B);

  (* EA / device2 sealing receipt for poll workers *)
  event sealing_receipt(HB, C, T, B);
  event ea_receipt(HB, C, T, B);

  (* EA publishes PBB entry *)
  new Count : bitstring;
  event pbb_entry(B, T, C, Count);

  0
```

You can later split this into separate `Device1`, `Device2`, and `EA` processes; for court verifiability, it's enough that all three evidence sources are tied to the same `(HB, C, T, B)`.

---

4. Judge's acceptance rule as a process

We now encode the *logical* condition under which a judge would accept the evidence. In ProVerif, we approximate this by a process that waits for all three evidence events and then emits `judge_accepts`.

```
(* Judge process: if all three evidence sources line up, accept *)
let Judge =
  in dummy_channel();  (* never used; just to make it a process *)
  (* In practice, we use events, not channels, so we model judge as: *)
  (!
    (* Wait for a triple of consistent events *)
    (* ProVerif doesn't "wait" on events, so we encode judge_accepts
       as emitted by the EA once it knows all three exist. For now,
       we simply emit judge_accepts after pbb_entry, assuming EA
       only does so when receipts are consistent. *)
    0
  ).

(* For now, we just emit judge_accepts at the end of the honest run *)
```

```
process
  (* Honest run from above *)
  new ID : id;
  new r  : bitstring;
  let HB = H(ID, r) in
  new T  : time;
  new B  : booth;
  new C  : choice;
  let FV = F(HB, C) in

  event cast_final(HB, C, T, B);
  event voter_receipt(HB, FV, T, B);
  event sealing_receipt(HB, C, T, B);
  event ea_receipt(HB, C, T, B);
  new Count : bitstring;
  event pbb_entry(B, T, C, Count);

  (* EA/Judge decide to accept this triple *)
  event judge_accepts(HB, C, T, B);

  0
```

This looks trivial now, but it gives us a hook for the **correspondence query**; later, when you add adversarial EA or malicious voters, you'll constrain when `judge_accepts` can be emitted.

---

5. Court-verifiability queries

5.1. Soundness: no false positives

If the judge accepts a case for HBID `h`, then there really was a unique final cast for `h`.

```
(* Soundness of court verifiability *)
query inj-event(judge_accepts(h, c, t, b)) ==> inj-event(cast_final(h, c, t,
b)).
```

This is the formal version of your "soundness of court verifiability":

"A malicious voter cannot falsely claim a fraud vote to disrupt the election if judges can perform the simple function calculation by themselves then compare it with the EA receipt and PBB records."

If ProVerif finds no attack, then **no trace exists** where `judge_accepts` happens without a matching unique `cast_final`.

5.2. EA cannot fabricate receipts without a real vote

EA should not be able to produce a sealing receipt and PBB entry for an HBID that never had a final vote.

```
query inj-event(ea_receipt(h, c, t, b)) ==> inj-event(cast_final(h, c, t,
b)).
```

Later, when you model a potentially malicious EA, this query will tell you whether the protocol structure (HBID generation, device logs, etc.) prevents such fabrication.

---

6. How to extend this skeleton

Once this basic model type-checks and ProVerif confirms the correspondences, you can:

- **Add an adversarial voter process** that:
  - Reuses an old receipt as if it were final.
  - Tries to guess HBIDs or fabricate `F(HB, C)` pairs.
- **Split EA and devices**:
  - Device2 emits `cast_final` and `sealing_receipt`.
  - EA emits `pbb_entry` only after seeing `sealing_receipt`.
- **Guard `judge_accepts`**:
  - Only emitted when all three events exist and are consistent (you can encode this by having EA emit `judge_accepts` only after both `sealing_receipt` and `pbb_entry` for the same tuple).

If you paste this into a `.pv` file, I can help you evolve it into a full model with explicit channels, an active attacker, and the absent-voting / non-inclusion scenarios from §4.4.1.

Nice—that's the point where it stops being abstract and becomes a real object you can poke at.

Right now, the skeleton I gave you has one important "practical" issue: ProVerif expects **one** top-level `process`, not two separate ones. So first, make the file syntactically clean, then run it, then start extending.

---

1. Make the .pv file a valid minimal model

Use something like this as a complete file:

```
(* Types *)
type id.
type hbid.
type choice.
type time.
type booth.
type fval.
type count.

(* Functions *)
fun H(id, bitstring): hbid.
fun F(hbid, choice): fval.

(* Events: ground truth *)
event cast_final(hbid, choice, time, booth).
event sealing_receipt(hbid, choice, time, booth).
event pbb_entry(booth, time, choice, count).

(* Events: what the court sees *)
event voter_receipt(hbid, fval, time, booth).
event ea_receipt(hbid, choice, time, booth).
event judge_accepts(hbid, choice, time, booth).

(* Main process: one honest run *)
process
  new ID : id;
  new r  : bitstring;
  let HB = H(ID, r) in
  new T  : time;
  new B  : booth;
  new C  : choice;
  let FV = F(HB, C) in

  (* Ground truth and evidence events *)
  event cast_final(HB, C, T, B);
  event voter_receipt(HB, FV, T, B);
  event sealing_receipt(HB, C, T, B);
  event ea_receipt(HB, C, T, B);
  new N : count;
  event pbb_entry(B, T, C, N);

  (* Judge/EA decide to accept this triple *)
```

```
  event judge_accepts(HB, C, T, B);

  0

(* Queries *)

(* Soundness: judge_accepts implies a unique cast_final *)
query inj-event(judge_accepts(h, c, t, b)) ==> inj-event(cast_final(h, c, t,
b)).

(* EA cannot fabricate a receipt without a cast_final *)
query inj-event(ea_receipt(h, c, t, b)) ==> inj-event(cast_final(h, c, t,
b)).
```

Save that as, say, `court_verif.pv`.

---

2. Run ProVerif on it

From a terminal:

```
proverif court_verif.pv
```

You should see output like:

- `RESULT inj-event(judge_accepts(...)) ==> inj-event(cast_final(...)) is true.`
- `RESULT inj-event(ea_receipt(...)) ==> inj-event(cast_final(...)) is true.`

If there's a syntax error, ProVerif will point to the line/column—fix until it parses cleanly.

Right now, the model is trivial (no attacker, one honest run), so both queries *should* be proved.

---

3. First nontrivial extension: expose receipts to an attacker

To make the court-verifiability queries meaningful, you want to let an attacker see receipts and try to trick the judge.

Add a public channel and outputs:

```
free net : channel.

process
  new ID : id;
  new r  : bitstring;
  let HB = H(ID, r) in
```

```
  new T  : time;
  new B  : booth;
  new C  : choice;
  let FV = F(HB, C) in

  event cast_final(HB, C, T, B);
  event voter_receipt(HB, FV, T, B);
  out(net, (HB, FV, T, B));          (* voter shows receipt on a public
channel *)

  event sealing_receipt(HB, C, T, B);
  event ea_receipt(HB, C, T, B);
  out(net, (HB, C, T, B));           (* EA's sealing receipt visible too *)

  new N : count;
  event pbb_entry(B, T, C, N);
  out(net, (B, T, C, N));            (* PBB is public by design *)

  event judge_accepts(HB, C, T, B);

  0
```

Run ProVerif again:

- If the queries still hold, you've just shown that **even with full visibility of receipts and PBB**, no trace exists where `judge_accepts` happens without a unique `cast_final`.

---

4. Next step: add a "cheating voter" process

Once this works, the next interesting move is to add a second process that tries to cheat—for example, reusing an old receipt or fabricating tuples—and see whether ProVerif can find an attack that violates the soundness query.

If you'd like, you can paste your current `court_verif.pv` (with any errors you hit), and we can iterate it into a version where a malicious voter is explicitly modeled and the court-verifiability query becomes genuinely nontrivial.